

FSM 리팩토링 포트폴리오

Controller 중심 State 구조를 조합 가능한 Player FSM으로 전환한 사례

핵심 결과

State 전환 책임을 분리하고, 이동 표현을 Blend Tree로 이관하며, Animator 제어 대상을 실제 Prefab 구성과 일치시켰다.

1. 리팩토링 이전 구조

초기 FSM은 State를 시작 시점에 생성해 Dictionary에 보관하고 전환 시 재사용했다. 이는 State 전환마다 객체를 생성하는 방식의 GC와 전환 비용을 피하기 위한 선택이었다.

당시 `AgentController`는 State 생성·보관·전환과 이동, 입력, 전투, Animator 접근을 함께 제공했다. State 기반은 `AgentStateBase<T>`와 `GroundedAgentStateBase`로 계층화되어 있었고, `AgentStateMachine<IAgentState>`가 현재 State와 전환을 관리했다.

State는 `\_agent.ChangeState(...)`를 직접 호출했다. 따라서 전이 조건, 전환 실행, Controller 접근 경로가 하나의 State 안에서 결합되었고, 기능 구현을 Handler로 분리한 뒤에도 실제 의존성이 잘 드러나지 않았다.

구분	리팩토링 전	핵심 문제
상태 기반	`AgentStateBase` + `GroundedAgentStateBase`	generic/지상 전용 기반이 State 구현에 전파
상태 전환	State 내부에서 `_agent.ChangeState` 직접 호출	전이 조건과 전환 실행이 결합
상태 머신	`AgentStateMachine`	Controller-StateMachine-State에 책임이 분산
Controller	기능 접근과 State 관리를 폭넓게 담당	State별 필요 기능이 누적되어 God Object 성격 확대

초기 흐름

State → AgentController → Input / Movement / Combat / Animator

State → _agent.ChangeState → AgentStateMachine

2. 첫 구조 전환

첫 변경은 단순히 Factory를 추가한 작업이 아니었다. State 기반, 전환 책임, 상태 머신 소유권을 먼저 단순화하고, 이후 Factory가 필요한 의존성과 전이 규칙을 조합하도록 바꾼 단계였다.

구분	리팩토링 전	1차 변경	상태
상태 기반	`AgentStateBase` + `GroundedAgentStateBase`	비제네릭 `AgentStateBase` 하나로 통합	적용
상태 전환	State에서 `_agent.ChangeState` 직접 호출	`ITransitionRule` 순차 평가 후 `OnTransition` 발행	부분 적용
상태 머신	`AgentStateMachine`	`AgentController`가 `_currentState` 직접 보유	적용

전환 책임의 변화

`AgentStateBase` 통합으로 공통 생명주기와 전이 규칙 등록 지점을 하나로 모았다. `AgentStateMachine` 제거 후에는 Controller가 State Dictionary와 `\_currentState`를 직접 소유해 현재 State의 소유자와 전환 실행 지점을 명확히 했다.

`ITransitionRule`은 State가 전환을 직접 실행하는 대신 전환 가능 여부와 다음 State를 선언한다. State는 Rule을 순차 평가하고 `OnTransition`을 발행하며, Factory가 이를 Controller의 전환 처리와 연결한다. Player FSM에는 적용됐지만 Monster와 기존 State 이행은 남아 있어 부분 적용으로 기록한다.

Controller → StateFactory → State(필요한 Handler / Input / Animator capability만 주입)

ITransitionRule → State.OnTransition → AgentController 전환

3. 설계 판단과 사용자 체감 개선

Factory는 State가 필요한 Handler·Input·Animator capability만 생성자로 받게 했다. 이로써 State의 의존성이 코드 시그니처에 드러나고, Controller 전체를 통해 기능에 접근하는 경로를 줄였다.

`ITransitionRule`은 이동 입력, 착지, 피격, Animation Event 종료 같은 조건을 State의 실행 코드에서 분리한다. 각 Factory가 필요한 Rule을 조합하므로 전이 우선순위와 책임을 한 곳에서 검토할 수 있다.

Idle/Move의 빈번한 상태 왕복은 `GroundedState`와 `MoveSpeed` 1D Blend Tree로 대체했다. 반대로 AttackType은 타격 시점·종료 이벤트가 다른 이산 행동이므로 Blend Tree가 아니라 Attack Sub State Machine에서 선택한다.

4. Animator 디버깅 사례

FSM이 GroundedState에 진입했지만 Animator가 전환하지 않는 문제는 로직만의 문제가 아니었다. Prefab의 Root와 Hero child에 Animator가 이중 구성되어 있었고, PlayerAnimator가 실제 렌더링 대상이 아닌 Root를 제어하고 있었다.

두 Player Prefab에서 PlayerAnimator 참조를 Hero child Animator로 통일하고 Root Animator를 제거했다. 이후 Animator parameter 계약은 AgentAnimator의 공통 Hit·Death와 PlayerAnimator의 Grounded·Jump·Fall·Attack·MoveSpeed·AttackType으로 분리했다.

5. 검증과 다음 단계

검증 속	완료된 구성	다음 수동 검증
FSM	Factory 구성, Type Dictionary, Rule 연결	입력 반복 시 State 전환 순서
Animator	Grounded Blend Tree, parameter 등록, Hero 바인딩	State와 표시 Animation 동기화
Jump	소비형 요청과 하강 Fall 조건	플랫폼 가장자리·공중 재입력
Attack	AttackEndTransition 이벤트 기반 복귀	홀드 반복과 공중 AttackType 3

Monster 확장은 보류한다. 현재 문서는 Player 직접 입력 기반 구현을 완료 기준으로 삼으며, Monster는 실제 capability와 Animator parameter를 표로 정의한 뒤 전용 Factory와 Animator 구조로 이행할 예정이다.